Lab #08 1st November, 2025

Instructions for Submission

1. File Naming and Submission:
   a. Create a **ZIP file** containing all your solution files.
   b. Name the ZIP file as `lab08_<ERP>.zip`, where `<ERP>` is your ERP.
   c. Ensure that the ZIP file contains:
      i. C++ source files
      ii. No unnecessary files (such as executables, temporary files, or editor configs)
   d. Do not include `TEXT(.txt)` files.
2. Code Neatness:
   a. Ensure your code is **well-formatted** and easy to read. Use proper **indentation** and **meaningful variable names**.
   b. Include appropriate **comments** to explain the logic where necessary (especially for functions and complex sections of the code).
   c. **Add clear explanations where required (or specified)** to make the logic of your program easy to follow.
3. Code Compilation and Functionality:
   a. Ensure your code compiles without errors or warnings. You should be able to compile and run your code on any system with a standard C++ compiler (e.g., GCC, Clang, MSVC).
   b. Double-check that the functionality of the program meets the requirements outlined in the task.
4. Additional Instructions:
   a. Do not use any libraries or features that have not been covered in the course material, unless specifically instructed.
   b. Make sure your code adheres to the principles of good software development (e.g., minimal repetition, and no hard-coding).
   c. If you encounter any issues or require clarifications, reach out before the submission deadline.
5. Deadline:
   a. Ensure that you submit the assignment on time. Late submissions will not be accepted.
   b. Double-check that the zip file contains all required files before submitting.
6. Plagiarism Policy:
   a. The work submitted must be entirely your own. Any code or content copied from others (including online sources, classmates or AI) will result in an automatic zero for the task.

Lab Tasks

Task 1: Basics

Part 1

Create a function **`celsiusToFahrenheit`** that converts a temperature from Celsius to Fahrenheit. This function should take a temperature in Celsius as input, convert it using the formula Fahrenheit = (Celsius * 9/5) + 32, and return the result. The user should be prompted to enter a temperature in Celsius, which will then be passed to this function.

Function Header: **`double celsiusToFahrenheit(double celsius)`**

Part 2

Create a function **`power`** that takes two integers as input, a base and an exponent, and calculates the base raised to the power of the exponent. The function should handle cases where the exponent is zero or negative. Prompt the user to input both the base and exponent values and display the result of the calculation.

Function Header: **`double power(double base, double exponent)`**

Part 3

Create a function **`factorial`** that returns the factorial of a given number `n'. Ask the user to input a positive integer and display the factorial value returned by the function.

Function Header: **`int factorial(int n)`**

Submit all these parts in one file **T1.cpp** and in the main function use all the above functions mentioned above, in the output each function should be clearly labelled.

Task 2: Vectors in Functions

Part A

Design a function that modifies the original data vector directly to achieve maximum performance and memory efficiency, essential when dealing with very large datasets.

1. **Function Signature:** Write a function named **`scale_vector_in_place`**.
   - It must take a **std::vector<double>&data** as its first parameter (using **non-constant reference**).
   - It must take a **double factor** as its second parameter.
   - It must **return void**.
2. **Implementation:** Loop through the data vector and multiply every element by the factor.
3. **Demonstration:** In your `main()` function, create a vector V1 (e.g., {1.0,2.0,3.0}). Call **`scale_vector_in_place`** on V1. Print the contents of V1 *after* the function call to prove the original vector was **permanently changed**.

Part B

Design a function that guarantees the original data vector remains untouched, prioritizing data safety and avoiding side effects.

1. **Function Signature:** Write a function named **`get_scaled_copy`**.
   - It must take a **const std::vector<double>&data** as its first parameter (using **constant reference**).
   - It must take a **double factor** as its second parameter.
   - It must **return a *new* std::vector<double>**.
2. **Implementation:** Create a **new local vector copy**. Loop through the input data vector, calculate the scaled value for each element, and add it to the **new local vector**. Return this new vector.
3. **Demonstration:** In your `main()` function, create a vector V2 identical to V1. Call **`get_scaled_copy`** and store the result in a new vector V3. Print the contents of V2 (to prove it's **unchanged**) and V3 (to show the new result).

Task 3: Code Refactoring and Documentation

A software development firm recently fired a developer, and as a form of revenge, he deliberately rewrote the system in a messy, unstructured way. All logic is contained within the main() function, variable names are inconsistent, user prompts are vague or misleading, and there is no documentation. This has made it extremely difficult for other developers to understand, maintain, or extend the system.

Your task is to:

- Analyze the given unstructured code.
- Break it down into meaningful functions.
- Write proper documentation for each function using the format below.

```
/**
 * @brief <short summary>
 *
 * @param <name> <description>
 * @return <description if applicable>
*/
```

Be sure to identify logical groupings of operations (input, processing, and output). Refactor the code so that each function has a single clear purpose. Add brief and relevant comments where appropriate.

Additional Notes:

- The original code is poorly formatted and uses inconsistent indentation, spacing, and brace placement.
- Variable names are not meaningful or consistent, and user prompts are vague or unhelpful.
- There are multiple repeated or similar blocks of code that could be refactored into reusable functions.
- Loops and logic are often repeated unnecessarily across different control paths.

Provided Code:

The unstructured program to be refactored is included in the file legacy_code.cpp, which is attached with the lab PDF. You must not change the program's functionality — only its structure, readability, clarity, and user interface if needed.